

Yıldız Teknik Üniversitesi
Elektrik-Elektronik Fakültesi
Bilgisayar Mühendisliği Bölümü

BLM2642 Bilgisayar Mühendisleri için Diferansiyel Denklemler
Gr: 1
Prof.Dr.Mehmet Fatih Amasyalı
Ödev 1

İsim: ismail Orhan
No : 23011081
Mail: ismail.orhan@std.yildiz.edu.tr
Tel: 05464426349

Ön Bilgi

Bu çalışma, verilen soru–cevap çiftlerinin “iyi” veya “kötü” olarak sınıflandırılması amacıyla kurulmuş bir regresyon temelli modeli kapsamaktadır. Ödev kapsamında hem veri üretim süreci hem de eğitim/test aşamaları detaylı biçimde uygulanmıştır. Soru ve cevapların anlamsal temsilleri için Turkish E5 Large, veri üretimi için ise Turkish Gemma 9B T1 modelleri kullanılmıştır. Bu temsiller birleştirilerek tek katmanlı tanh çıkışlı bir model eğitilmiş, ardından farklı başlangıç ağırlıkları için GD, SGD ve Adam algoritmaları zaman, iterasyon ve başarı kriterlerine göre karşılaştırılmıştır.

Optimizasyon sürecinin davranışını incelemek amacıyla ağırlık güncellemeleri kayıt altına alınmış ve bu değerler T-SNE yöntemiyle iki boyutta görselleştirilmiştir. Ayrıca eğitim sürecini desteklemek üzere model performansını etkileyen faktörler üzerine ek araştırmalar yapılmıştır.

Projeye ait kodların bağlantısı: Optimization_odev.zip

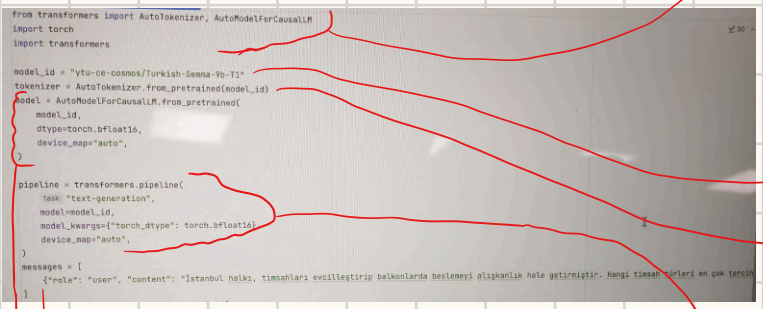
Ödevde istenen sunum kısmı için modelin çalışma prensibini, kullandığım yöntemleri ve elde edilen sonuçları anlattığım bir video hazırlanmış ve YouTube’a yüklenmiştir:

Video bağlantısı: Diferansiyel Denklemler Optimizasyon Ödev

Rapor içerisindeki önemli noktalar, ara hesaplamalar ve grafiklerle ilgili yorumlar süreci daha iyi kavrayabilmek için el yazısı ile alınmış notlarla desteklenmiştir. Bu notlar, çalışma boyunca karşılaşılan sorunları, denenen farklı yaklaşımları ve çıkarılan sonuçları daha net açıklamaktadır.

Metin üretme → Bir modelin girdi olarak aldığı kısa bir talimatı kullanarak yeni ve anlamlı bir yazı oluşturmasıdır.

Cosmos Kod



```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
import transformers

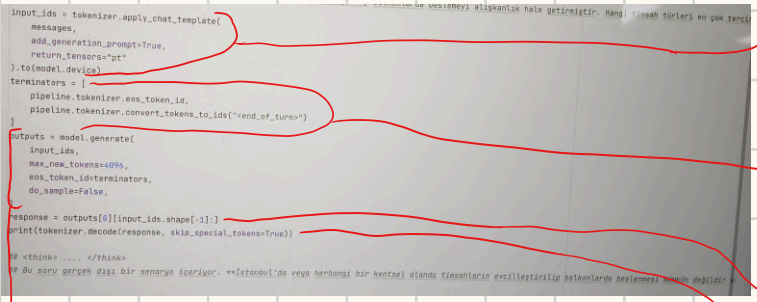
model_id = "ytu-ce-cosmos/Turkish-Gemma-9b-T1"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    dtype=torch.bfloat16,
    device_map="auto",
)

pipeline = transformers.pipeline(
    task="text-generation",
    model=model_id,
    model_kwargs={"torch_dtype": torch.bfloat16},
    device_map="auto",
)
messages = [
    {"role": "user", "content": "İstanbul halkı, timsahları evcilleştirip balkonlarda beslemeyi alışkanlık hale getirmiştir. Hangi timsah türleri en çok tercih
]
```

- transformers → Hugging Face modellerini kullanmanı sağlar
- AutoTokenizer → Metni modele verebilmek için parçalar (token'e) eder)
- AutoModelForCausalLM → Dil modeli (örneğin Gemma) yüklemek için
- torch → model hesaplamalarını yapar.
- Hugging Face üzerindeki modelin adı, Tokenizer ve model buradan yüklenecek.
- Modelin yerdi ağırlık parçalarını yapısını indirir. Yazdığı tüm metinleri sayılara çevirmekten sorumludur.
- Pipeline bir kolaylık kısayoludur. Pipeline tokenizer, model, generate ayarlarını tek bir obje haline getirir. (Yani kullanmanda bilirsin)
- Model sohbet türü eğitildiğine mesajları belli bir formatta görmesi gerekir.
- Model indirip belleğe yükler. Model'i çalışmaya hazır hale getirir.

dtype = torch.bfloat16 → Bellekte daha az yer kaplayan bir sayı formatı. Büyük modeller için gerekli

device_map = "auto" → Modeli GPU varsa GPU'ya, yoksa CPU'ya otomatik yerleştirir.

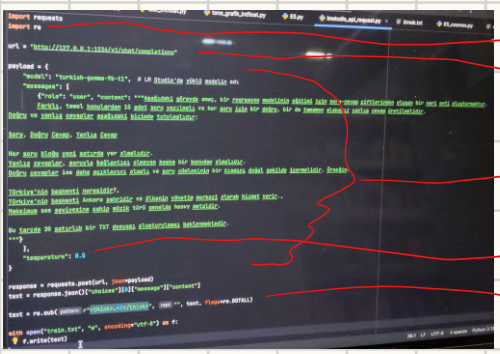


```
input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt"
).to(model.device)
terminators = [
    pipeline.tokenizer.eos_token_id,
    pipeline.tokenizer.convert_tokens_to_ids("<end_of_turn>")
]
outputs = model.generate(
    input_ids,
    max_new_tokens=4096,
    eos_token_id=terminators,
    do_sample=False,
)
response = outputs[0][input_ids.shape[-1]:]
print(tokenizer.decode(response, skip_special_tokens=True))

# <think> ..... </think>
# Bu soru gerçek dışı bir senaryo içeriyor. **İstanbul'da veya herhangi bir kentsel alanda timsahların evcilleştirilip balkonlarda beslenmesi mümkün değildir.**
```

- Tokenizer ile sohbet formatını input'a çevirme. Modele uygun sohbet formatını yazar. Metni tokenlere çevirir.
- return_tensors="pt" → tokenize sonrası veriyi PyTorch tensörüne çevirir.
- .to(model.device) → girdi tensörünü modelin çalıştığı cihaza geçirir.
- Model bu ID'lerden birini görünce üretimi durdurur.
- Sadece modelin ürettiği yeni kısımları alıyor.
- Modelden çıktıyı üretme. Model burada saf üretim yapıyor, pipeline kullanmadık. Yani hepsini manuel yazdık
- input_ids → modele verdiğin giriş
- max_new_tokens = 4096 → en fazla 4096 token üret
- eos_token_id = terminators → bu tokenlerden biri gelince üretimi kes
- do_sample = false → Rastgelelik kapalı, model en olası çıktıyı seçer
- Tokenleri string'e çevirir.
- özel işaretleri atar (<end_of_turn> gibi)

API ile metin üretme Kod



```
import requests
import re

url = "http://127.0.0.1:1234/v1/chat/completions"

payload = {
    "model": "turkish-gemma-9b-t1",
    "messages": [
        {"role": "user", "content": "
Hayır, timsahlar evcilleştirilip balkonlarda beslenemez.
Bunun birkaç nedeni var:
Güvenlik: Timsahlar vahşi hayvanlardır ve saldırgan olabilirler.
Boyut: Timsahlar büyüdüklerinde çok büyük olurlar ve balkonlarda yaşamaları mümkün değildir.
Yasal: Timsah beslemek birçok ülkede yasa dışıdır.
Sağlık: Timsahlar hastalık taşıyabilirler.
Çevre: Timsahlar doğal yaşam alanlarında yaşamalıdırlar.
Bu nedenlerle, timsahları evcilleştirip balkonlarda beslemek mümkün değildir.
"
    },
    "temperature": 0.0
}

response = requests.post(url, json=payload)
text = response.json()["choices"][0]["message"]["content"]

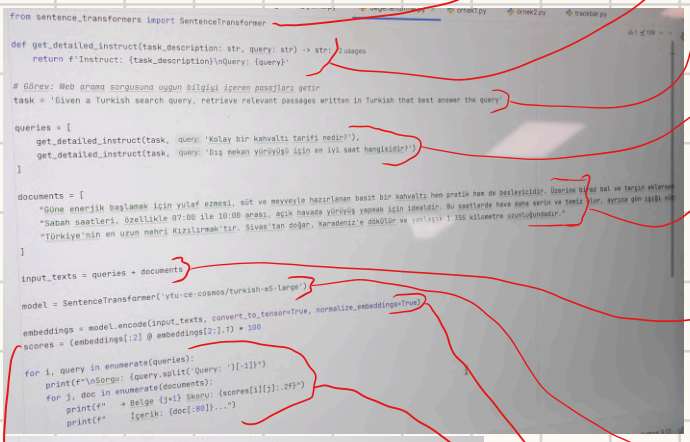
text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL)

with open("train.txt", "w", encoding="utf-8") as f:
    f.write(text)
```

- Regular Expression (Düzenli ifadeler) kütüphanesidir. Metin desenlerini aramak, eşleştirmek ve değiştirmek için kullanılır.
- İsteğin gönderileceği adrestir.
- Sunucuya gönderilen talimatlardır.
- Yaratıcılık / rastgelelik seviyesidir. 0.0 değeri, yanıtın hem yaratıcı olmasını hem de tutarlı kalmasını sağlar.
- think etiketini ve içindekileri bulur ve "" ile değiştirir yani temizler.

Metin değerlendirme → Metinlerin (örneğin bir cümle, paragraf, dökümanın) vektör temsillerini oluşturur. Bu temsiller sayesinde metinler arası "anlamsal benzerlik", "anlamsal arama" gibi işlemler yapılabilir.

Cosmos Kod



Cümleleri sayısal vektörlere dönüştüren bir model tiri.

bu fonksiyon sorguya ek bir görev açıklaması ekliyor

Çıktı şöyle oluyor:

```
Instruct: Given a Turkish search query...
Query: Kolay bir kahvaltı tarifi nedir?
```

Yani modelin daha iyi embedding üretmesi sağlanıyor.

bu görev embedding modeline bağlam verir:
"Sorguya uygun türkçe pasajları bulacak şekilde düşün"

2 adet sorgu oluşturuluyor her biri: Görev + Soru formatında birleştiriliyor.

her belge embedding yapılacak bir metin. Bu belgeler arasından sorgulara en uygun olanı bulunacak.

Sorgular (2 adet) + Belgeler (3 adet) → toplam 5 adet metin, Embedding modeli hepsini tek seferde encode ediyor. (not embedding, bir cümlenin, belgenin, veya kelimenin sayılardan oluşan bir vektörel(liste) temsilidir)

bu model, türkçe için optimize edilmiş, çıktı olarak yüksek boyutlu vektörler (embedding) üretir.

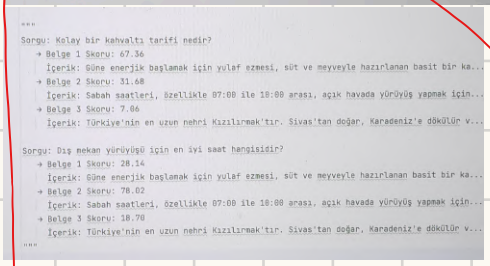
Tüm metinleri embedding'e çevirme encode → metni sayısal embedding'e çevirir (her metin → 1024 boyutlu (örnek) bir vektör)

convert_to_tensor=True → PyTorch tensör olarak döner (hesaplama hızlı olur)

normalize_embeddings=True → vektörleri birim uzunluğuna çeker

Sonuç:

```
embeddings[0] → Sorgu 1'in vektörü
embeddings[1] → Sorgu 2
embeddings[2] → Belge 1
embeddings[3] → Belge 2
embeddings[4] → Belge 3
```



embeddings[:2] → ilk 2 vektör (sorgular) embeddings[2:] → sonraki 3 vektör (belgeler)

@ → matris çarpımı = cosine similarity (normalize edilmişse)

T → transpoz

*100 → skoru yüzdeye çevirme

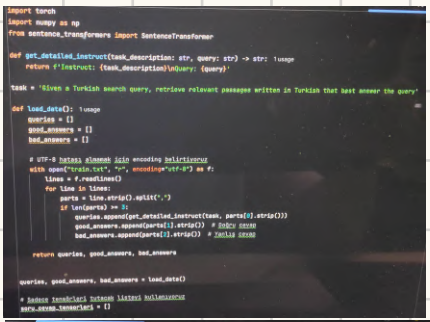
Sonuç:

```
scores =
[
[s11, s12, s13],   # Query 1 için her belgenin skoru
[s21, s22, s23]    # Query 2 için her belgenin skoru
]
```

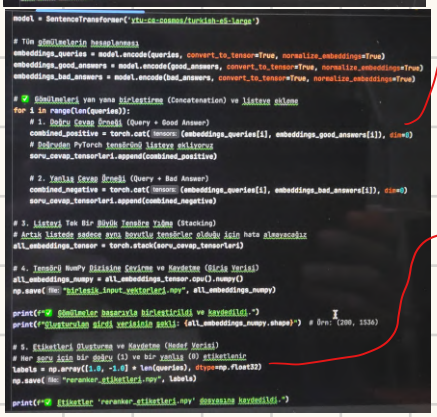
her sorgu için her belgenin benzerlik skoru yazılıyor ve ilk karakter belge önizlemesi veriyor.

Not: Kodda sorguları ve belgeler önce birleştirildi sonrada ayırarak işlem yürütüldü. Hiç birleştirmedende yapabilirsin

Benim Kodum



Temel olarak txt dosyasında kayıt ettiğim soru ve cevapları ayırıp bir diziye kaydediyor. Sonra ES model ile vektöre dönüştürüp ve soruları ve cevapları ayrı ayrı birleştirip bir .npy dosyasına kaydediliyor.



dim=0 → Tensörler dikey olarak, yani satır sayısını artırarak birleştirilir.

dim=1 → Tensörler yatay olarak, yani sütun sayısını artırarak birleştirilir.

[1.0, -1.0] * len(queries) → [1.0, -1.0, 1.0, -1.0 ...]

dtype=np.float32 → Sayıların hafızada nasıl saklanacağını belirtir. float32 → Sayısal çok hassastır, derin öğrenme modellerinde sıkça kullanılır.

```python
import numpy as np
import time
from grafik_cizdirme import grafik
from tsne_grafik import Tsne_grafik

y = np.load("reranker_etiketleri.npy")  #(100, 1) # Hedef etiketler (1.0: Doğru, -1.0: Yanlış). Toplam 100 örnek.
X = np.load("train_vektor.npy")  #(100, 2048) # Birleştirilmiş girdi gömülmeleri (Sorgu + Cevap). 100 örnek, 2048 boyut.

Y = y.reshape(-1,1)

W = np.random.uniform(low=-0.1, high=0.1, size=(X.shape[1], 1))
```

Burada y'yi yeniden düzenlemek gerekiyor çünkü normalde y'nin boyutu (100,) yani bir dizi ama bize bir matris olmasını istiyoruz bu hal ile sorunlar çıkarıyor. Sorunu şöyle söyleyebilirim tahmin_y - Y yaparken Y eğer bir dizi olursa çıkan sonuç (100,1) olması gerekirken (100,100) boyutunda oluyor ve sonuç yanlış çıkıyor.

-1 ile 1 arasında (X.shape[1],1) (yani (2048,1) boyutunda) random bir matris üretir. (X.shape[1] → (100, 2048))

Gradient Descent → bir fonksiyonun en küçük değerini bulmak için kullanılan temel bir optimizasyon yöntemidir.

```python
def gradient_descent(X, Y, W_baslangic, lr = 0.1, epochs = 100):
    W = W_baslangic.copy()
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []
    #W_values = []

    # Bu döngü, tüm veri seti üzerinde 1000 kez eğitim yapar.
    for epoch in range(epochs):
        hata = np.mean((np.tanh(X @ W) - Y) ** 2) #(100,2048) * (2048,1) - (100,1)
        hata_epochs.append([epoch, hata])
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if(np.tanh(X @ W)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru/100])
        basari_time.append([time.time() - baslangic_zamani, dogru/100])
        #W_values.append(W.T.copy())

        # İleri Yayılım: Tahmin = tanh(Girdi @ Ağırlıklar)
        tahmin_y = np.tanh(X @ W)  # (100, 1) = (100, 2048) @ (2048, 1)

        # Geri Yayılım: Gradyan Bileşenleri
        d_loss = tahmin_y - Y  # (100, 1) # Hata (Fark)
        d_tanh = 1 - tahmin_y ** 2  # (100, 1) # tanh Aktivasyonunun Türevi
        delta = d_loss * d_tanh  # (100, 1) # Zincir Kuralının İlk Kısmı (Eleman bazlı çarpım)

        # Ağırlık Gradyanı (∂L/∂w): Gradyan = X.T @ Delta (Matris Çarpımı)
        d_W = X.T @ delta #  (2048, 1) = (2048, 100) @ (100, 1)

        # Ağırlık Güncelleme: W = W - (Öğrenme Hızı * Gradyan / Örnek Sayısı)
        W -= (lr * d_W) / len(Y)  # (2048, 1)

    # W_values = np.array(W_values)
    # W_values_reshaped = W_values.reshape(W_values.shape[0], W_values.shape[2])
    # eski = np.load("W_GD.npy")
    # yeni = np.vstack([eski, W_values_reshaped])
    # np.save("W_GD.npy", yeni)
    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)
```

time.time() → Şu anki zamanı saniye cinsinden, 1 ocak 1970 tarihinden itibaren geçen süre olarak verir.

fonksiyonun içine verilen değişkenler fonksiyonun içinde değişse bile dışında aynı kalır ama fonksiyonun içine verdiğimiz bir dizi veya matris fonksiyonun içinde değiştirilirse fonksiyonun dışında da değişir. Eğer bir bunu yazmayıp direk W'yi kopyalasaydık W burada optimize edilirdi ve diğer yerlerde optimize olan W gibi olarak alırdı.

başarı için gereken formül olan $\frac{doğru}{tüm\ durum}$ uygulanıyor. gerçek y ile tahmin y'nin işaretleri doğru ise doğru oluyor.

burada W.T.copy() demeliyim çünkü ben normal kaydettiğimde her güncellemede W_value'nin bütün değerleri de değişiyor tek bir değer çıkıyor. Bu yüzden W.T.copy() diyerek güncelleme ile W_value değerlerinin güncellenmesini engelliyorum.

Ben her seferinde W.T'yi yani (1,2048)'lik bir matrisi diziye kaydediyorum ve en sonda döngüden çıktığımda (N, 1, 2048)'lik matris ortaya çıkıyor ben bunu (N, 2048) boyutuna dönmeliyim

Listeyi diziye çevirip düzgün bir matris yapısına kavuşmak ve daha rahat işlem yapmak için np.array() yapıyoruz Bu sayede kolay kaydetme, alt ile grafik kolay bir şekilde çizme vb. birçok faydası var.

np.save() diyince dosya ya eklene yerine üzerine yazar ve doğrudan eklemek için de bir fonksiyon yoktur bizde dosyadaki değerleri alır yeni değer ile birleştirir ve onu dosyaya yazarız.

SGD → Stochastic Gradient Descent, klasik gradient descent'in daha hızlı bir versiyonudur.

```python
def SGD(X, Y, W_baslangic, lr = 0.1, epochs = 100, batchSize = 10):
    W = W_baslangic.copy()
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []
    #W_values = []

    for epoch in range(epochs):
        # Hata Hesaplama
        hata = np.mean((np.tanh(X @ W) - Y) ** 2)
        hata_epochs.append([epoch, hata])  # Epoch'a göre çizim
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if(np.tanh(X @ W)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru/100])
        basari_time.append([time.time() - baslangic_zamani, dogru/100])
        #W_values.append(W.T.copy())

        index = np.random.permutation(len(X))
        X_shuffled = X[index]
        Y_shuffled = Y[index]

        for i in range(0, len(X), batchSize):
            xBatch = X_shuffled[i:i + batchSize]
            yBatch = Y_shuffled[i:i + batchSize]

            # --- Backward ---
            tahmin = np.tanh(xBatch @ W)
            d_loss = tahmin - yBatch
            d_tanh = 1 - tahmin ** 2
            delta = d_loss * d_tanh

            # Gradyan Hesaplama ve KRİTİK BÖLME İŞLEMİ
            d_W = xBatch.T @ delta
            d_W /= batchSize

            W -= lr * d_W

    # W_values = np.array(W_values)
    # W_values_reshaped = W_values.reshape(W_values.shape[0], W_values.shape[2])
    # eski = np.load("W_SGD.npy")
    # yeni = np.vstack([eski, W_values_reshaped])
    # np.save("W_SGD.npy", yeni)
    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)
```

Burada np.random.permutation(n) diyerek 0'dan n'e kadar olan sayıları rastgele bir şekilde bir dizi hale geldi. Yani index → [7, 11, 1, 3, n-1, 0 ...]

karıştırılan bir dizi yi başka bir diziye verirse o dizide karıştırılan dizi sırasında yer değiştirir. Yani:

X → [0, 1, 2, 3 ...]
index → [2, 1, 3, 0]
X[index] → [2, 1, 3, 0]

0'dan len(x)'e kadar batchSize aralıklarla i değişken:

dizi[i:i+batchSize] yani i'den i+batchSize olan kısmı başka bir diziye kaydet. Yani: X_shuffled → [8, 7, 3, 2, 6, 1, 4] batchSize → 3, döngü → 2 xBatch → [2, 6, 1]

mini-batch'te gradyan ortalamasını almalıyım. Batch boyutu büyüdükçe güncelleme oranlısız bir şekilde büyür. Öğrenme dengesiz olur. Bu önemli.

ADAM → gradient descent'in daha akıllı ve stabil çalışan bir versiyonudur.

Mantığı iç parçadan oluşur:

- Momentum kullanır → geçmiş gradyanların dikkate alıp güncellemeyi yumuşatır.
- Adaptif öğrenme oranı kullanır → her parametre için ayrı öğrenme oranı hesaplar.
- Bias correction uygular → Başlangıçtaki sapmalar düzeltir.

```
def ADAM(X, Y, W_baslangic, lr = 0.01, epochs = 100, batchSize = 10):  1 usage
    W = W_baslangic.copy()
    B1 = 0.9
    B2 = 0.999
    e = 1e-8
    m = np.zeros_like(W)
    v = np.zeros_like(W)
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []
    #W_values = []

    t = 0  # Genel güncelleme adımı sayacı

    for epoch in range(epochs):
        # Hata Hesaplama
        hata = np.mean((np.tanh(X @ W) - Y) ** 2)
        hata_epochs.append([epoch, hata])
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if(np.tanh(X @ W)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru/100])
        basari_time.append([time.time() - baslangic_zamani, dogru/100])
        #W_values.append(W.T.copy())

        index = np.random.permutation(len(X))
        X_shuffled = X[index]
        Y_shuffled = Y[index]

        for i in range(0, len(X), batchSize):
```

0'lardan oluşan bir matris oluştur, boyutu W'nin boyutları kadar olsun

```
        for i in range(0, len(X), batchSize):
            t += 1  # Her batch için adımı artır
            xBatch = X_shuffled[i:i + batchSize]
            yBatch = Y_shuffled[i:i + batchSize]

            # --- Backward ---
            tahmin = np.tanh(xBatch @ W)
            d_loss = tahmin - yBatch
            d_tanh = 1 - tahmin ** 2
            delta = d_loss * d_tanh

            d_W = xBatch.T @ delta
            g = d_W / batchSize  # Mini-Batch Gradyan Ortalaması

            # ADAM Güncelleme
            m = B1 * m + (1 - B1) * g
            v = B2 * v + (1 - B2) * (g * g)

            m_hat = m / (1 - B1 ** t)
            v_hat = v / (1 - B2 ** t)

            W -= lr * m_hat / (np.sqrt(v_hat) + e)
    # W_values = np.array(W_values)
    # W_values_reshaped = W_values.reshape(W_values.shape[0], W_values.shape[2])
    # eski = np.load("W_ADAM.npy")
    # yeni = np.vstack([eski, W_values_reshaped])
    # np.save("W_ADAM.npy", yeni)

    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)
```

np.sqrt(v_hat) = v_hat**(1/2) ama ilk ifade büyük boyutlu matrisler için daha hızlı çalışır bu yüzden avantajlıdır.

```
gradient_descents = []
SGDs = []
ADAMs = []

for i in range(1):

    gradient_descents.append(gradient_descent(X, Y, W))
    SGDs.append(SGD(X, Y, W))
    ADAMs.append(ADAM(X, Y, W))

    W = np.random.uniform(low=-0.1, high=0.1, size=(X.shape[1], 1))

grafik(gradient_descents, SGDs, ADAMs)
```

belirli sayıda farklı w değerleri ile başlayan algoritmaları kaydetme

```
import matplotlib.pyplot as plt
import numpy as np


def grafik(gradient_descents, SGDs, ADAMs):  2 usages
    SIZE = len(gradient_descents)
    FONTSIZE = 5

    hata_epochs_GDs = []
    hata_time_GDs = []
    basari_epochs_GDs = []
    basari_time_GDs = []
    hata_epochs_SGDs = []
    hata_time_SGDs = []
    basari_epochs_SGDs = []
    basari_time_SGDs = []
    hata_epochs_ADAMs = []
    hata_time_ADAMs = []
    basari_epochs_ADAMs = []
    basari_time_ADAMs = []

    for i in range(SIZE):

        hata_epochs_GD, hata_time_GD, basari_epochs_GD, basari_time_GD = gradient_descents[i]
        hata_epochs_SGD, hata_time_SGD, basari_epochs_SGD, basari_time_SGD = SGDs[i]
        hata_epochs_ADAM, hata_time_ADAM, basari_epochs_ADAM, basari_time_ADAM = ADAMs[i]

        hata_epochs_GDs.append(hata_epochs_GD)
        hata_time_GDs.append(hata_time_GD)
        basari_epochs_GDs.append(basari_epochs_GD)
        basari_time_GDs.append(basari_time_GD)
        hata_epochs_SGDs.append(hata_epochs_SGD)
        hata_time_SGDs.append(hata_time_SGD)
        basari_epochs_SGDs.append(basari_epochs_SGD)
        basari_time_SGDs.append(basari_time_SGD)
        hata_epochs_ADAMs.append(hata_epochs_ADAM)
        hata_time_ADAMs.append(hata_time_ADAM)
        basari_epochs_ADAMs.append(basari_epochs_ADAM)
        basari_time_ADAMs.append(basari_time_ADAM)

    plt.figure(figsize=(16, 9))  # figürü büyüt
    plt.subplots_adjust(
        left=0.04, right=0.98,  # kenar boşluklarını daralt
        top=0.95, bottom=0.05,
        hspace=0.25, wspace=0.15  # subplot aralarını küçült
    )

    for i in range(SIZE):
        plt.subplot(SIZE,4, i * 4 + 1)
        plt.title(f"Hata - Epochs W_{i + 1}")
        plt.xlabel("Epochs")
        plt.ylabel("Hata")
        plt.plot(hata_epochs_GDs[i][:, 0], hata_epochs_GDs[i][:, 1], color="red")
        plt.plot(hata_epochs_SGDs[i][:, 0], hata_epochs_SGDs[i][:, 1], color="blue")
        plt.plot(hata_epochs_ADAMs[i][:, 0], hata_epochs_ADAMs[i][:, 1], color="green")
        plt.legend(["Gradient Descent", "Stochastic Gradient Descent", "ADAM"], loc="upper right", fontsize=FONTSIZE)
        plt.grid()

        plt.subplot(SIZE,4,i * 4 + 2)
        plt.subplot(SIZE,4,i * 4 + 3)
        plt.title(f"Başarı - Epochs W_{i + 1}")
        plt.xlabel("Epochs")
        plt.ylabel("Başarı")
        plt.plot(basari_epochs_GDs[i][:, 0], basari_epochs_GDs[i][:, 1], color="red")
        plt.plot(basari_epochs_SGDs[i][:, 0], basari_epochs_SGDs[i][:, 1], color="blue")
        plt.plot(basari_epochs_ADAMs[i][:, 0], basari_epochs_ADAMs[i][:, 1], color="green")
        plt.legend(["Gradient Descent", "Stochastic Gradient Descent", "ADAM"], loc="lower right", fontsize=FONTSIZE)
        plt.grid()

        plt.subplot(SIZE,4,i * 4 + 4)
        plt.title(f"Başarı - Zaman W_{i + 1}")
        plt.xlabel("Zaman")
        plt.ylabel("Başarı")
        plt.plot(basari_time_GDs[i][:, 0], basari_time_GDs[i][:, 1], color="red")
        plt.plot(basari_time_SGDs[i][:, 0], basari_time_SGDs[i][:, 1], color="blue")
        plt.plot(basari_time_ADAMs[i][:, 0], basari_time_ADAMs[i][:, 1], color="green")
        plt.legend(["Gradient Descent", "Stochastic Gradient Descent", "ADAM"], loc="lower right", fontsize=FONTSIZE)
        plt.grid()

    plt.tight_layout()
    plt.show()
```

grafiğin fiziksel boyutunu belirler genişlik:16, yükseklik:9, yani geniş ekran bir grafik oluşturuyor

grafik içindeki boşlukları ayarlar. left:0.04 → 0.04* figür genişliği kadar boşluk bırak, top:0.95 → üstten %5 boşluk

hspace:0.25 → subplot yüksekliğinin %25'i kadar boşluk ekle

figür içinde belirli bir alt grafik (subplot) seçmek için kullanılır. SIZE → kaç satır olacağı, 4 → kaç sütun olacağı, $i*4+n$ kaçıncı subplot'un seçileceği

Tüm grafik düzeni içinde (i*4+1). sıradaki subplot'u aktif hale getiriyor. Bu noktadan sonra yapılan çizimler o bölüme yapılır.

hata_epochs_GDs[i][:,0] → i'nci denemenin epoch değerleri (x ekseni), hata_epochs_GDs[i][:,0] → i'nci denemenin hata değerleri (y ekseni)

color:"red" → kırmızı ile çiz, plt.plot(...) → bu iki veriyi bir çizgiyle çiz.

Grafiğin arka planına ızgara çizgileri ekler.

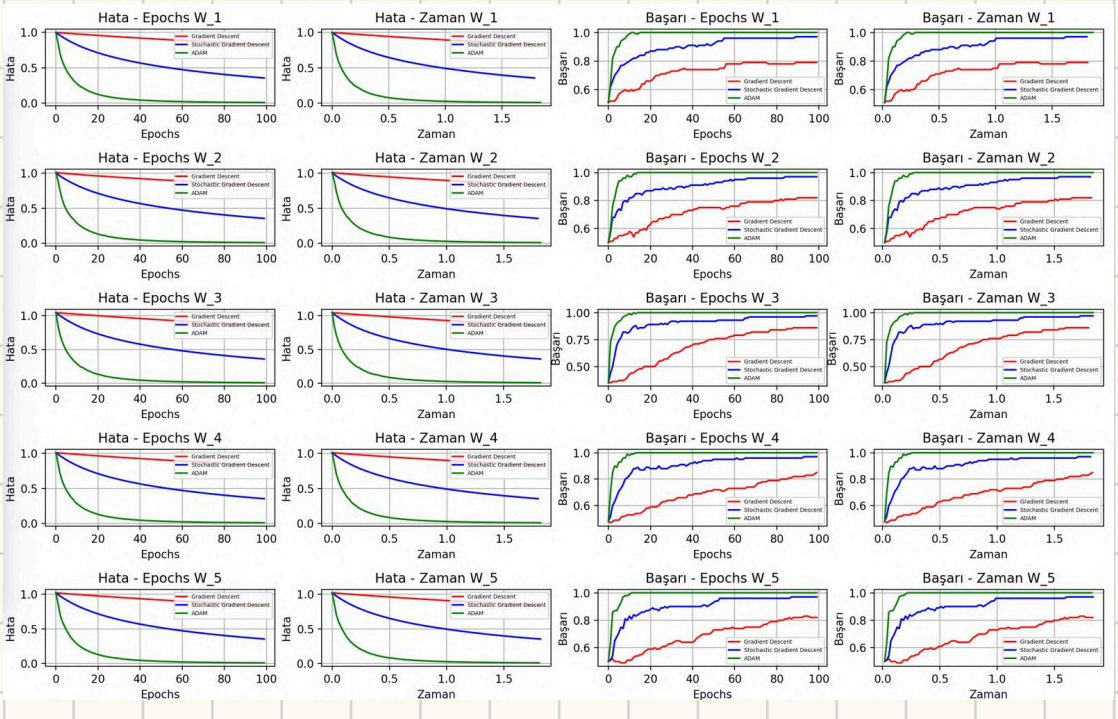
["Gradient..."] → her çizginin ismi (legend'de görünecek metinler), loc:"upper right" → Legend kutusunun grafiğin sağ üst köşesine yerleşmesini sağlar

fontsize:.. → yazı boyutunu ayarlar.

figürdeki subplot'ların birbirine veya kenarlara fazla yaklaşmasını önler ve otomatik olarak boşlukları optimize eder.

grafiği ekranda gösterir. plt.show() çağrılana kadar çizilen grafikler ekranda görünmez.

plt.subplots_adjust()'ın otomatik hali gibi düşünebilirsin Yukarıda plt.subplots_adjust()'ı silebilirsin.



TSNE → yüksek boyutlu veriyi 2D veya 3D'ye indirip görselleştirmek için kullanılan bir yöntemdir.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE


def Tsne_grafik():  1 usage
    FONTSIZE = 7
    W_GD = np.load("W_GD.npy")
    W_SGD = np.load("W_SGD.npy")
    W_ADAM = np.load("W_ADAM.npy")

    tsne = TSNE(
        n_components=2,
        perplexity=5,
        learning_rate=200,
        max_iter=1000,
        random_state=10,
    )

    W_GD_2D = tsne.fit_transform(W_GD)
    W_SGD_2D = tsne.fit_transform(W_SGD)
    W_ADAM_2D = tsne.fit_transform(W_ADAM)

    plt.figure(figsize=(16, 9))

    colors = ["red", "blue", "green", "orange", "purple"]
    plt.subplot( *args: 3, 1, 1)
    for i in range(5):
```

t-SNE algoritması için bir nesne oluşturuyor. n_components=2 → veriyi 2 boyuta indir (2D görselleştirme)

perplexity = 5 → komşuluk sayısını etkiler, küçük değer → yerel yapı, büyük değer → global yapı

learning_rate = 200 → optimizasyon hızı max_iter = 1000 → maksimum iterasyon sayısı

random_state = 10 → tekrar üretilebilir sonuç için rastgelelik sabiti

t-SNE algoritmasını uygular ve 2048 boyutlu W_ADAM'i 2 boyutlu veriye düşürür.

W_ADAM_2D → artık her veri noktası 2D koordinat olarak temsil edilir.

```python
for i in range(5):
    W_value = W_GD_2D[i * 100: i * 100 + 100]
    plt.plot( *args: W_value[:,0], W_value[:, 1], color=colors[i], label=f"W_GD_{i}", lw=1.5)
    plt.scatter(W_value[0, 0], W_value[0, 1], color=colors[i], marker='o')
    plt.scatter(W_value[-1, 0], W_value[-1, 1], color=colors[i], marker='x')
    plt.title("T-SNE 2D Gradient Descent")
    plt.xlabel("T-SNE Boyut 1")
    plt.ylabel("T-SNE Boyut 2")
plt.grid()
plt.legend(loc="upper right", fontsize= FONTSIZE)

plt.subplot( *args: 3, 1, 2)
for i in range(5):
    W_value = W_SGD_2D[i * 100: i * 100 + 100]
    plt.plot( *args: W_value[:,0], W_value[:, 1], color=colors[i], label=f"W_SGD_{i}", lw=1.5)
    plt.scatter(W_value[0, 0], W_value[0, 1], color=colors[i], marker='o')
    plt.scatter(W_value[-1, 0], W_value[-1, 1], color=colors[i], marker='x')
    plt.title("T-SNE 2D SGD")
    plt.xlabel("T-SNE Boyut 1")
    plt.ylabel("T-SNE Boyut 2")
plt.grid()
plt.legend(loc="upper right", fontsize= FONTSIZE)
```

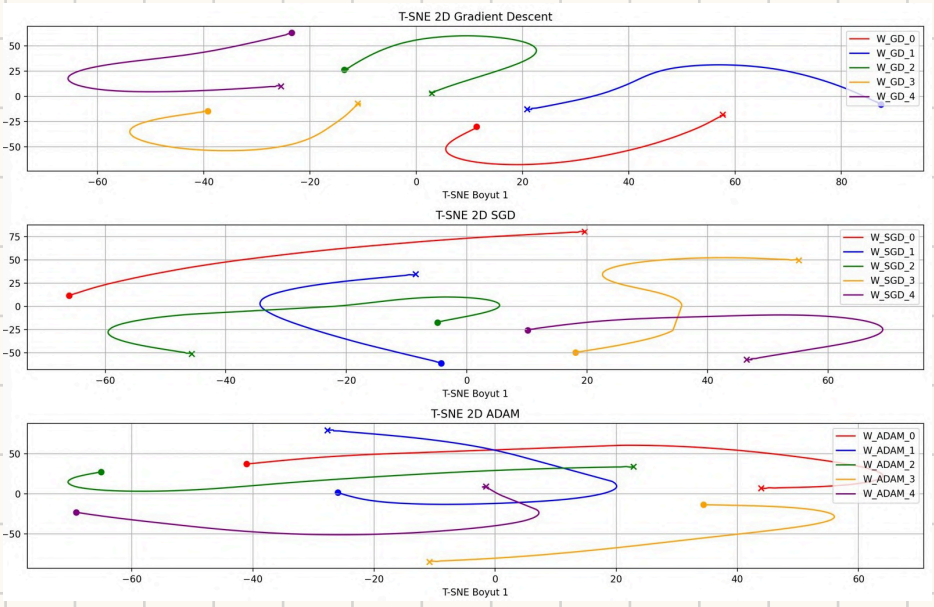
label: ... → legend için isim, lw = 1.5 → çizgi kalınlığı

grafiğe tek bir noktayı eğer W_value[0,0] → x koordinatı, W_value[0,1] → y koordinatı

marker = "o" → noktanın şekli (daire)

```python
plt.subplot( *args: 3, 1, 3)
for i in range(5):
    W_value = W_ADAM_2D[i * 100: i * 100 + 100]
    plt.plot( *args: W_value[:,0], W_value[:, 1], color=colors[i], label=f"W_ADAM_{i}", lw=1.5)
    plt.scatter(W_value[0, 0], W_value[0, 1], color=colors[i], marker='o')
    plt.scatter(W_value[-1, 0], W_value[-1, 1], color=colors[i], marker='x')
    plt.title("T-SNE 2D ADAM")
    plt.xlabel("T-SNE Boyut 1")
    plt.ylabel("T-SNE Boyut 2")
plt.grid()
plt.legend(loc="upper right", fontsize= FONTSIZE)

plt.tight_layout()
plt.show()
```



Test verisi → Model eğitim verisiyle eğittim, ağırlık (W) değerlerini kaydettim ve bu değerleri test verisi üzerinde kullanarak performansı değerlendirdim.

```python
import numpy as np
import time
from grafik_çizdirme import grafik

W_GD = np.load("W_GD.npy") #(500, 2048)
W_SGD = np.load("W_SGD.npy")
W_AG = np.load("W_AG.npy")
W_ADAM = np.load("W_ADAM.npy")

y = np.load("reranker_etiketleri.npy")  #(100, 1) # Hedef etiketler (1.0: Doğru, -1.0: Yanlış). Toplam 100 örnek.
X = np.load("test_vektor.npy")  #(100, 2048) # Birleştirilmiş girdi gömülmeleri (Sorgu + Cevap). 100 örnek, 2048 boyut.

Y = y.reshape(100,1)
```

```python
def gradient_descent(X, Y, W_GD, W_index, epochs = 100) : 1 usage
    W = W_GD[W_index * 100: W_index * 100 + 100]
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []

    for epoch in range(epochs):
        W_new = W[epoch].reshape(-1, 1)
        hata = np.mean((np.tanh(X @ W_new) - Y) ** 2) #(100,2048) * (2048,1) - (100,1)
        hata_epochs.append([epoch, hata])
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if(np.tanh(X @ W_new)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru/100])
        basari_time.append([time.time() - baslangic_zamani, dogru/100])

    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)
```

```python
def SGD(X, Y, W_SGD, W_index, epochs = 100): 1 usage
    W = W_SGD[W_index * 100: W_index * 100 + 100]
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []

    for epoch in range(epochs):
        W_new = W[epoch].reshape(-1, 1)
        hata = np.mean((np.tanh(X @ W_new) - Y) ** 2)  # (100,2048) * (2048,1) - (100,1)
        hata_epochs.append([epoch, hata])
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if (np.tanh(X @ W_new)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru / 100])
        basari_time.append([time.time() - baslangic_zamani, dogru / 100])

    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)
```

```python
def ADAM(X, Y, W_ADAM, W_index, epochs = 100): 1 usage
    W = W_ADAM[W_index * 100: W_index * 100 + 100]
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []

    for epoch in range(epochs):
        W_new = W[epoch].reshape(-1, 1)
        hata = np.mean((np.tanh(X @ W_new) - Y) ** 2)  # (100,2048) * (2048,1) - (100,1)
        hata_epochs.append([epoch, hata])
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if (np.tanh(X @ W_new)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru / 100])
        basari_time.append([time.time() - baslangic_zamani, dogru / 100])

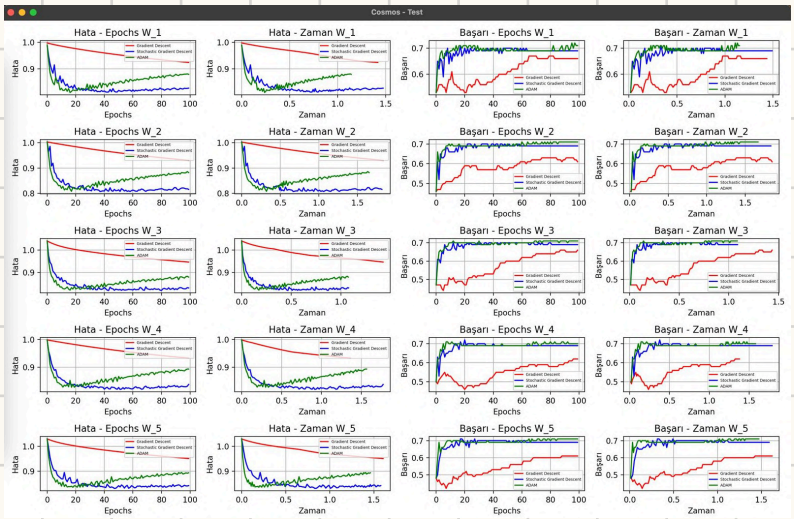
    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)
```

```python
gradient_descents = []
SGDs = []
AGs = []
ADAMs = []

for i in range(5):

    gradient_descents.append(gradient_descent(X, Y, W_GD, i))
    SGDs.append(SGD(X, Y, W_SGD, i))
    AGs.append(AdaGrad(X, Y, W_AG, i))
    ADAMs.append(ADAM(X, Y, W_ADAM, i))

grafik(gradient_descents, SGDs, AGs, ADAMs)
```

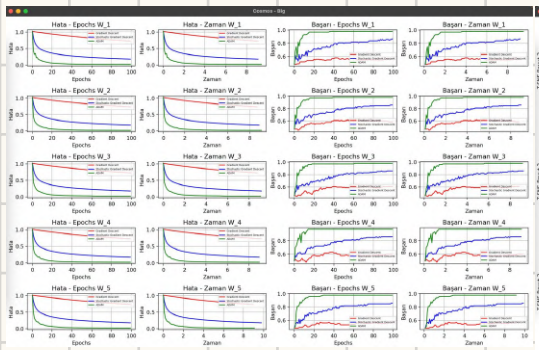


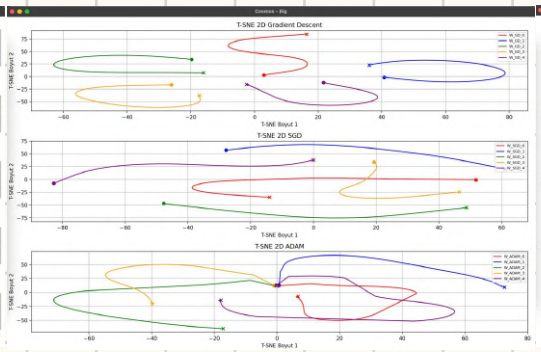
Bonuslar

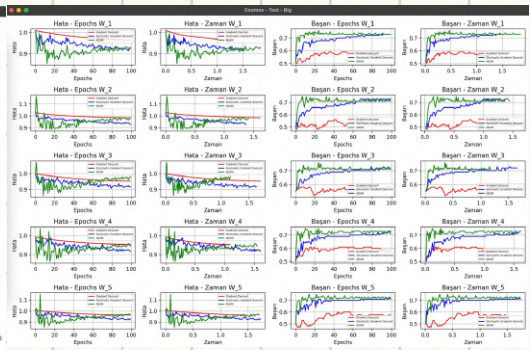
1. Eğitim kümesinin büyüklüğünü performansa etkisini araştırılması.

Bu çalışmada, eğitim kümesinin boyutunun model performansına etkisi incelenmiştir. Başlangıçta model yalnızca 50 örnekten oluşan küçük bir eğitim setiyle eğitildi. Daha sonra eğitim verisi genişletilerek 250 örneklik daha büyük bir set oluşturuldu. Her iki durumda da değerlendirme için aynı 50 örneklik test veri seti kullanıldı.

Küçük eğitim setiyle model, sınırlı örnek çeşitliliği nedeniyle daha düşük genelleme başarısı gösterdi. Eğitim seti 250 örneğe çıkarıldığında, modelin daha fazla örüntü öğrenebildiği ve test performansının belirgin şekilde iyileştiği gözlemlendi. Bu sonuç, eğitim verisinin büyüklüğünün modelin doğruluğu ve kararlılığı üzerinde önemli bir etkiye sahip olduğunu göstermektedir.







2. Başka optimizasyon algoritmalarının karşılaştırmaya dahil edilmesi.

- Dahil edilen algoritma Ada Grad algoritmasıdır.

Ada Grad → Her parametreye ayrı öğrenme oranı veren, seyrek veride güçlü ama uzun vadede yavaşlayan bir optimizasyon yöntemidir.

```python
def AdaGrad(X, Y, W_baslangic, lr = 0.1, epochs = 100, batchSize = 10):
    W = W_baslangic.copy()
    e = 1e-8
    G = np.zeros_like(W)
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []
    # W_values = []

    for epoch in range(epochs):
        # Hata Hesaplama
        hata = np.mean((np.tanh(X @ W) - Y) ** 2)
        hata_epochs.append([epoch, hata])  # Epoch'a göre çizim
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if(np.tanh(X @ W)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru/100])
        basari_time.append([time.time() - baslangic_zamani, dogru/100])
        # W_values.append(W.T.copy())

        index = np.random.permutation(len(X))
        X_shuffled = X[index]
        Y_shuffled = Y[index]

        for i in range(0, len(X), batchSize):
            xBatch = X_shuffled[i:i + batchSize]  #(batchSize, 2048)
            yBatch = Y_shuffled[i:i + batchSize]  #( batchSize, 1)
```

AdaGrad'ın gerektirdiği birikmiş kare gradyan matrisini oluşturmak için W boyutunda sıfır matris başlatıyoruz.

```python
            # --- Backward ---
            tahmin = np.tanh(xBatch @ W)
            d_loss = tahmin - yBatch
            d_tanh = 1 - tahmin ** 2
            delta = d_loss * d_tanh

            # Gradyan Hesaplama ve KRİTİK BÖLME İŞLEMİ
            d_W = xBatch.T @ delta
            d_W /= batchSize

            G += d_W ** 2 #(2048, 2048)

            W -= lr * d_W / (np.sqrt(G) + e)  #(2048, 1)

    # W_values = np.array(W_values)
    # W_values_reshaped = W_values.reshape(W_values.shape[0], W_values.shape[2])
    # eski = np.load("W_AG.npy")
    # yeni = np.vstack((eski, W_values_reshaped))
    # np.save("W_AG.npy", yeni)
    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)

def ADAM(X, Y, W_baslangic, lr = 0.01, epochs = 100, batchSize = 10):...
```

AdaGrad'ın temel olarak formülü şudur:

g = gradyan

$G += g^{**}2$

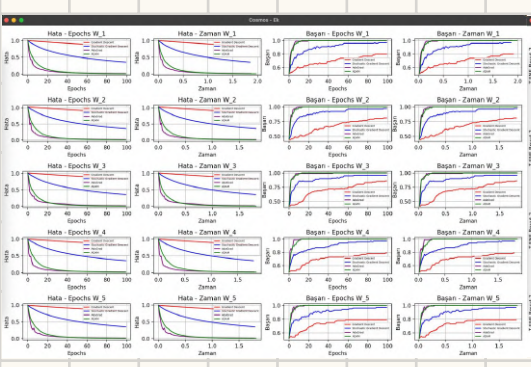
$W -= lr * g / (\sqrt{G} + e)$

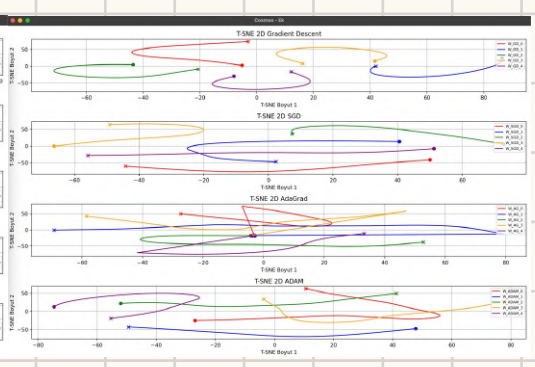
Test için

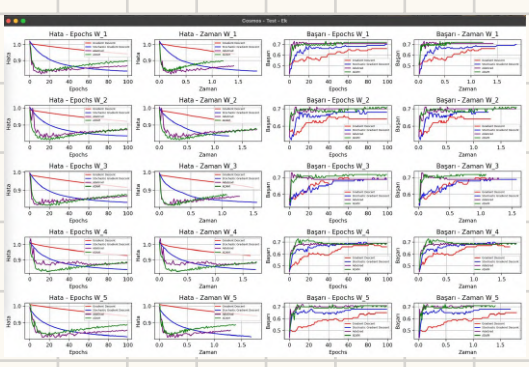
```python
def AdaGrad(X, Y, W_SGD, W_index, epochs = 100): 1 usage
    W = W_SGD[W_index * 100: W_index + 100 + 100]
    baslangic_zamani = time.time()
    hata_time = []
    hata_epochs = []
    basari_epochs = []
    basari_time = []

    for epoch in range(epochs):
        W_new = W[epoch].reshape(-1, 1)
        hata = np.mean((np.tanh(X @ W_new) - Y) ** 2)  # (100,2048) * (2048,1) - (100,1)
        hata_epochs.append([epoch, hata])
        gecen_zaman = time.time() - baslangic_zamani
        hata_time.append([gecen_zaman, hata])
        dogru = 0
        for j in range(100):
            if (np.tanh(X @ W_new)[j][0] * Y[j][0] > 0):
                dogru += 1
        basari_epochs.append([epoch, dogru / 100])
        basari_time.append([time.time() - baslangic_zamani, dogru / 100])

    return np.array(hata_epochs), np.array(hata_time), np.array(basari_epochs), np.array(basari_time)
```







3. Başka anlamsal temsil modellerin karşılaştırmaya dahil edilmesi.

Bu çalışmada iki model karşılaştırılmıştır. İlk model olarak türkçe odaklı ytu-ce-cosmos/turkish-e5-large kullanılmıştır. Karşılaştırma için ikinci model olarak çok dilli bir yapı sunan intfloat/multilingual-e5-large değerlendirmeye dahile edilmiştir. Her iki model de aynı metin benzerliği ve sınıflandırma testlerinde kullanılmıştır.

Testler sonucunda türkçe odaklı modelin, anlamca yakın ifadeleri ayırt etmede daha tutarlı ve yüksek doğruluklu sonuçlar ürettiği gözlemlenmiştir. Çok dilli model ise daha geniş dil bilgisine sahip olmasına rağmen türkçe metinlerde zaman zaman daha yüzeysel gömmeler üretmiş ve benzer cümleler ayırmada türkçe modele göre daha zayıf performans göstermiştir. Genel olarak, türkçe'ye özel eğitilmiş modelin türkçe veri üzerinde belirgin biçimde daha yüksek benzerlik doğruluğu ve daha istikrarlı embedding çıktısı verdiği sonucuna ulaşılmıştır.

